SIMATS ENGINEERING

Department of Computer Science and Engineering

CSA15 Cloud Computing and Big Data Analytics

CO3 AT1 - Guided Cloud Access Experiments Workbook

Course Outcome	CO3 - Implement cloud-based solutions using cloud storage, web APIs, and platform services for real-world applications.
Unit Alignment	Unit 3 - Access to Cloud: cloud access, platforms, web applications, APIs, storage models, SAN, serverless, edge/fog, collaboration, security and development workflow.
Faculty	Dr C. Rajesh Babu, Professor, Department of CSE
Employee ID	SSETSCS415
Submission Mode	Individual PDF evidence through Google Classroom and GitHub repository link.

How This Workbook Works

Stage	Student Action
Learn	Read the short technical note before touching the tool.
Do	Perform the guided action using the recommended tool.
Observe	Check what changed on the screen or in the output.
Record	Paste screenshots, URLs, tables, command outputs or diagram references.
Explain	Write the technical reasoning in your own words.
Extend	Apply the same concept to the capstone product continued from CO2 AT3.
Submit	Upload the completed evidence to GitHub and Google Classroom.

Important Safety Rule

Screenshots must not reveal passwords, private keys, API tokens, payment details, personal account recovery information or sensitive user data. Crop or mask sensitive values before submission.

Student and Faculty Details

Student Name	D.S.THARUNIYA PRIYA	**Register Number**	192511317
Class / Section		**Slot**	B
Course	CSA15 Cloud Computing and Big Data Analytics	**Assessment**	CO3 AT1
Capstone Title	PulsePay:Cloud based fake payment Screenshot detection	**Submission Date**	05-08-2026
Faculty	Dr C. Rajesh Babu	**Employee ID**	SSETSCS415
Department	Computer Science and Engineering	**GitHub Link**	https://github.com/tharu-ui/Cloud-Computing-and-BDA/tree/main/PulsePay/CO3-AT1-Cloud-Access-Experiments

Tool Links

Tool	Purpose	Link
Firebase Console	Cloud storage and project evidence	https://console.firebase.google.com/
Postman	REST API request-response evidence	https://www.postman.com/downloads/
JSONPlaceholder	Safe public API for GET/POST practice	https://jsonplaceholder.typicode.com/
ReqRes	Safe public API for request-response practice	https://reqres.in/
draw.io	Architecture and workflow diagrams	https://app.diagrams.net/
GitHub	Repository evidence submission	https://github.com/
GitHub Pages	Simple hosted web access evidence	https://pages.github.com/

Experiment 1: Cloud Storage Model Comparison and Selection

Original CO3 Question	Create and document a cloud storage comparison between object, block, and file storage. Select the best storage model for backups, VM disks, and shared enterprise documents.
Tools	Firebase Console, Firebase Storage, draw.io, browser, GitHub.
CO2 AT3 Continuity	Use the same capstone title and infrastructure direction already defended in CO2 AT3. Do not repeat VM sizing; focus on cloud service access.

Learn

A cloud application may store files, database records, VM disks, images, logs and shared documents. Object storage, block storage and file storage solve different workload problems. Firebase Storage is used here as a beginner-friendly object storage console because students can create a project, open a storage bucket, upload a file, observe metadata and reason about access control from the browser.

Do

☐ Begin in the browser at https://console.firebase.google.com/. Sign in with the account approved for academic work. After the console opens, use Add project or Create a project. Give the project a course-friendly name such as csa15-co3-regno. If Google Analytics is shown, keep it disabled unless faculty instructs otherwise. Create the project and wait until the project dashboard opens. The screen to notice is the Firebase project overview page.

☐ In the left navigation area, open Build and choose Storage. If Storage is not yet enabled, choose the option to get started. Select the default bucket location shown by Firebase or the nearest available region. Use test access only for lab observation if the console requires an initial rule choice, and remember that production apps must use strict authenticated rules. The screen to notice is the storage bucket page.

☐ Create a small text file on the computer named student_backup.txt. Add two lines inside it: the student register number and the capstone title. Return to Firebase Storage and use Upload file. Select student_backup.txt and wait until it appears in the bucket list. The screen to notice is the object list showing the uploaded file.

☐ Open the uploaded file entry. Observe the metadata area: file name, file size, content type, storage path, creation/update time and access status. If a download URL or access token is visible, do not paste sensitive token values in the workbook. Record only the safe evidence: filename, type, size and storage path.

☐ Open draw.io and draw a three-part comparison diagram: object storage for uploaded files/backups, block storage for VM disks, and file storage for shared folders. Under the diagram, write where your capstone will use storage and why Firebase Storage represents object storage in this experiment.

Observe

Check whether the uploaded file is visible, whether metadata is displayed, and whether public/private access is controlled. Do not expose sensitive files.

Record Evidence

Evidence Item	Student Reference / Screenshot Number	Checkpoint
Cloud storage dashboard screenshot.	01-firebase-project-overview.png	[X] Completed
Uploaded object/file screenshot.	- blocked -	[ ] Completed
Metadata screenshot.	- blocked -	[ ] Completed
Completed storage model comparison table.	PulsePay_Storage_Comparison_Reference.png	[X] Completed
Capstone storage recommendation table.	Monitoring logging metric table.png	[x] Completed

Explain

Why is one storage model not suitable for all workloads? Justify using backup, VM disk and shared-folder examples, then extend the answer to your capstone product.

Student Explanation Space

No single storage model fits every workload because each is optimized for a different access pattern. Backups and uploaded files (like PulsePay's payment screenshots) suit object storage accessed as whole files via a simple key, with built-in durability. VM disks need block storage instead, since the OS reads/writes small fixed-size blocks constantly and needs low-latency, byte-level access. Shared enterprise documents need file storage, since multiple users must open, edit, and lock the same file concurrently through a folder hierarchy. For PulsePay, screenshots uploaded by merchants are a clear object storage case: each file is written once, read occasionally for verification, and never edited in place exactly what Firebase Storage / Cloud Storage was built for.

Extend to Capstone

Select the storage model required for your capstone product: user uploads, app images, database attachments, logs, backups or shared documents.

Capstone Title	PulsePay — Fake Payment Screenshot Detection Using Cloud Computing
CO2 AT3 Infrastructure Decision Carried Forward	Serverless-first architecture using Cloud Functions, Firestore, and Cloud Storage
Cloud Service / Access Layer Added in this Experiment	Object storage (Firebase/Cloud Storage) for merchant-uploaded payment screenshots
Risk or Limitation Identified	Test-mode storage rules allow open access; production must enforce authenticated, per-merchant access rules
Improvement for Prototype or Pilot	Add Firebase Security Rules restricting each merchant to only their own files, plus a lifecycle rule to auto-delete screenshots after the retention window

Submit

GitHub Folder	CO3_AT1/Experiment_1
Files to Upload	Completed PDF, diagrams, screenshots and any exported table.
Google Classroom Entry	Paste GitHub repository link and attach final PDF.

Experiment 2: REST API Workflow for a Cloud-Based Student Portal

Original CO3 Question	Develop a simple REST API workflow for a cloud-based student portal. Show request, response, authentication, and deployment flow.
Tools	Postman Web/Desktop, JSONPlaceholder or ReqRes, draw.io, browser, GitHub.
CO2 AT3 Continuity	Use the same capstone title and infrastructure direction already defended in CO2 AT3. Do not repeat VM sizing;

	focus on cloud service access.

Learn

REST API means Representational State Transfer Application Programming Interface. For beginners, think of it as a controlled doorway through which a client app asks a cloud service for data or sends data to be processed. A browser or mobile app sends a request to an API endpoint. The API validates the request, applies business logic, communicates with storage or database services, and returns a structured response. Postman is used because it lets students send requests, view status codes, inspect headers, read JSON responses and test API behavior without writing a full app first.

Do

☐ Open https://www.postman.com/ or the Postman desktop application. Sign in or create a free account if Postman asks for one. After the workspace opens, create a personal workspace or use the default workspace. Create a collection named CSA15_CO3_AT1_API so the evidence is organized.

☐ Open a new request tab. Keep the method as GET. Enter https://jsonplaceholder.typicode.com/users in the request URL field and send the request. Look at the response area. The expected observation is a successful status such as 200 OK and a JSON response containing user-like records.

☐ Read one object from the JSON response. Identify fields such as id, name, username, email, address and company. Record three fields in the workbook and explain what each field would mean in a student portal or capstone product.

☐ Create another request with method POST. Use https://jsonplaceholder.typicode.com/posts as the URL. Open the Body area, choose raw, select JSON, and enter a small JSON body such as {"title":"CSA15 cloud test","body":"API evidence","userId":1}. Send the request. The expected observation is a created/test response, often with status 201 or a simulated response.

☐ Open draw.io and draw the request-response flow. Show Browser/Mobile Client -> REST API Endpoint -> Backend Logic -> Database/Storage -> JSON Response. Add a small authentication gate before backend logic to show that real cloud applications should protect API access.

Observe

Verify whether the request returns status 200 or a valid test response. Identify JSON fields and explain what each field represents.

Record Evidence

Evidence Item	Student Reference / Screenshot Number	Checkpoint
Postman GET request screenshot.	01-postman-get-request.png	[x] Completed
Status code and response time screenshot.	01-postman-get-request.png	[x] Completed
JSON field identification table.	02-postman-post-request.png	[x] Completed
POST request body and response screenshot.	02-postman-post-request.png	[x] Completed
API workflow diagram.	API Workflow Diagram.png	[x] Completed

Explain

Why should a cloud application expose data through APIs instead of allowing users to directly access the database?

Student Explanation Space

Exposing data through an API instead of direct database access lets the backend enforce validation, authentication, and business logic before any data is touched — a client can't bypass these checks. It also means the database schema can evolve without breaking clients, since the API contract stays stable. For PulsePay, merchants should only submit a screenshot and receive a fraud-risk score through a controlled API endpoint, never query Firestore directly, which would expose other merchants' records and bypass the fraud-scoring logic entirely.

Extend to Capstone

Write two API endpoints for your capstone product and explain the input, backend action, storage access and response.

Capstone Title	PulsePay — Fake Payment Screenshot Detection Using Cloud Computing
CO2 AT3 Infrastructure Decision Carried Forward	Cloud Functions + API Gateway exposing REST endpoints, backed by Firestore and Cloud Storage
Cloud Service / Access Layer Added in this Experiment	POST /verify (input: screenshot + merchant ID; backend: OCR+ELA+ML scoring; storage: Firestore + Cloud Storage; response: JSON risk score) and GET /history/{merchantId} (input: merchant ID; backend: Firestore query; response: JSON list of past results)
Risk or Limitation Identified	Without rate limiting, a merchant could spam /verify and drive up OCR/Vision API cost
Improvement for Prototype or Pilot	Add per-merchant rate limiting and request quotas at the API Gateway layer

Submit

GitHub Folder	CO3_AT1/Experiment_2
Files to Upload	Completed PDF, diagrams, screenshots and any exported table.
Google Classroom Entry	Paste GitHub repository link and attach final PDF.

Experiment 3: Serverless Event Flow for Image Upload Processing

Original CO3 Question	Design a serverless event flow for image upload processing using object storage, function-as-a-service, and notification service.
Tools	draw.io, Firebase Storage evidence from Experiment 1, optional Pipedream/Make simulation, GitHub.
CO2 AT3 Continuity	Use the same capstone title and infrastructure direction already defended in CO2 AT3. Do not repeat VM sizing;

	focus on cloud service access.

Learn

Serverless computing allows a cloud platform to run code in response to events without the student managing the server directly. In this experiment, students do not need paid cloud function deployment. They design and document a correct event-driven flow. The key idea is: an event happens, a cloud function is triggered, the function performs a small task, and another cloud service records or sends the output.

Do

☐ Start with the uploaded file evidence from Experiment 1. Treat the upload as the event. For capstone mapping, choose one event such as image upload, booking creation, complaint submission, attendance scan, payment update or alert generation.

☐ In the workbook, write the event source in a short table. Mention who performs the action, what data is created, where it is stored, and what should happen automatically after the event.

☐ Design the cloud function behavior without writing production code. Name the function clearly, for example processUploadedImage, confirmBooking, validateAttendance or sendIssueAlert. Write what the function receives, what it checks, what it changes, and what output it produces.

☐ Decide the output service. It may be a notification, email, database status update, dashboard row, resized image, log entry or report update. Also decide what should happen if the function fails: retry, error log, faculty/admin alert or manual review.

☐ Draw the event flow in draw.io using this visual order: User Action -> Cloud Storage/Database Event -> Serverless Function -> Processing Decision -> Output Service -> Log/Monitoring. Paste the diagram and complete the trigger-action-output table.

Observe

Identify the trigger, processing function, output service, failure point and retry/alert requirement. If billing blocks real function deployment, submit a technically accurate simulation and architecture evidence.

Record Evidence

Evidence Item	Student Reference / Screenshot Number	Checkpoint
Event source evidence or simulated event table.	Trigger-Action-Output table.png	[x] Completed
Serverless flow diagram.	Serverless_Event_Flow.png	[x] Completed
Trigger-action-output table.	Trigger-action-output table.png	[x] Completed
Failure and retry note.	retry once, then flagged for manual review	[x] Completed
Capstone event mapping.	Extend to Capstone table.pngx	[x] Completed

Explain

Why is serverless suitable for upload/event processing? Mention one limitation also.

Student Explanation Space

Serverless suits upload/event processing because the workload is event-driven and bursty — a screenshot upload triggers processing, then nothing happens until the next upload, matching pay-per-invocation billing far better than a constantly running server. One limitation: cold-start latency — if the function hasn't run recently, the first invocation after an idle period takes longer, which could delay a merchant's result by a second or two.

Extend to Capstone

Apply the serverless idea to one real capstone workflow, such as image processing, booking confirmation, attendance validation, alert generation or report update.

Capstone Title	PulsePay — Fake Payment Screenshot Detection Using Cloud Computing
CO2 AT3 Infrastructure Decision Carried Forward	Cloud Functions triggered by Cloud Storage upload events
Cloud Service / Access Layer Added in this Experiment	Event-driven trigger: Cloud Storage upload → Cloud Function → Firestore write → dashboard update
Risk or Limitation Identified	Cold-start latency on first request after idle periods could delay fraud verdicts
Improvement for Prototype or Pilot	Consider a minimum-instance setting on the Cloud Function to reduce cold starts during business hours

Submit

GitHub Folder	CO3_AT1/Experiment_3
Files to Upload	Completed PDF, diagrams, screenshots and any exported table.
Google Classroom Entry	Paste GitHub repository link and attach final PDF.

Experiment 4: Edge and Fog Computing for Smart Traffic Management

Original CO3 Question	Compare edge computing and fog computing for a smart traffic management system. Include architecture, latency considerations, and deployment location.
Tools	draw.io, Google Sheets or Excel latency table, optional Wokwi/IoT screenshot only if faculty permits, GitHub.
CO2 AT3 Continuity	Use the same capstone title and infrastructure direction already defended in CO2 AT3. Do not repeat VM sizing; focus on cloud service access.

Learn

Cloud computing is powerful but may be physically distant from sensors and users. Edge computing places small decisions very near the data source. Fog computing introduces an intermediate distributed layer between edge devices and the central cloud. This experiment is an architecture reasoning activity; students are not required to build a real traffic

sensor. The correct output is a placement decision supported by latency and data-volume reasoning.

Do

☐ Use this scenario: traffic cameras and road sensors produce vehicle count, speed, congestion level and emergency vehicle alerts. Some actions must happen immediately at the signal, some can be handled at a regional traffic control unit, and some can be stored in the central cloud for analysis.

☐ Classify each task into edge, fog or cloud. Put instant signal switching and emergency alert detection near edge. Put junction-level aggregation and area coordination in fog. Put long-term storage, analytics dashboard and city-level reports in cloud.

☐ Open a spreadsheet and create a latency table with columns: task, data generated, decision deadline, suitable layer, reason. Use example decision deadlines such as 1 second for emergency signal response, 5-10 seconds for regional coordination and minutes/hours for reports. The exact numbers are reasoning aids, not measured network results.

☐ Draw the architecture in draw.io. Show sensors/cameras at the road, edge device at signal controller, fog node at regional traffic control, cloud platform for storage/analytics and dashboard for administrator. Use arrows to show data movement and labels to show why each layer is used.

☐ Connect the same idea to the capstone. If the capstone has live location, sensor, mobile offline, queue, emergency, QR attendance or fast alert behavior, identify what can be edge/fog/cloud. If the capstone does not need edge/fog, state that browser/mobile client -> cloud service access is sufficient and justify it.

Observe

Check whether time-critical actions are placed closer to data sources and whether long-term storage/analytics remain in the cloud.

Record Evidence

Evidence Item	Student Reference / Screenshot Number	Checkpoint
Task classification table.	Task classification table.png	[x] Completed
Latency reasoning table.	Latency reasoning table.png	[x] Completed
Edge-fog-cloud architecture diagram.	Edge-fog-cloud arch.png	[x] Completed
Deployment location justification.	Time-critical, safety-related decisions (emergency alerts, signal switching) are placed at the edge because even a few seconds of cloud round-trip latency is unacceptable when a vehicle is approaching an intersection. Fog nodes at regional traffic control handle coordination across multiple signals, which needs broader context than a single edge device has but still can't wait for a full cloud round-trip. The cloud is reserved for tasks with no real-time constraint — long-term storage, analytics, and city-wide reporting — where centralization and scale matter	[x] Completed

	more than speed.	
Capstone relevance note.	Refer explain text below	[x] Completed

Explain

Which part of the system should run at edge, fog and cloud? Justify with latency and data-volume reasoning.

Student Explanation Space

PulsePay has no hard real-time, location-sensitive requirement — a merchant uploading a screenshot can tolerate a second or two of latency without real-world safety consequence. So direct browser/mobile client → cloud service access is sufficient: OCR, forensics, and ML scoring all run centrally in Cloud Functions rather than at the edge/fog layer. A future real-time in-store QR scanning feature could benefit from edge-level pre-validation (e.g. checking image quality before upload), but core fraud-scoring logic stays cloud-only since it depends on server-side ML models and cross-merchant pattern data.

Extend to Capstone

If your capstone has location-sensitive, sensor-based, offline-first or fast-alert requirements, identify the edge/fog/cloud placement. If not applicable, justify why direct cloud access is sufficient.

Capstone Title	PulsePay — Fake Payment Screenshot Detection Using Cloud Computing
CO2 AT3 Infrastructure Decision Carried Forward	Fully cloud-centric serverless architecture, no edge/fog components
Cloud Service / Access Layer Added in this Experiment	Not applicable — direct client → cloud API access is sufficient
Risk or Limitation Identified	None currently; a future real-time scanning feature could introduce latency-sensitive needs
Improvement for Prototype or Pilot	If real-time scanning is added later, evaluate lightweight edge-side image quality checks before upload

Submit

GitHub Folder	CO3_AT1/Experiment_4
Files to Upload	Completed PDF, diagrams, screenshots and any exported table.
Google Classroom Entry	Paste GitHub repository link and attach final PDF.

Experiment 5: Cloud Application Deployment and Access Workflow

Original CO3 Question	Demonstrate a cloud application deployment workflow showing client access, browser interaction, web API call, storage access, and monitoring.
Tools	GitHub Pages as primary beginner path, Firebase Hosting as optional path, Postman, draw.io, GitHub repository,

	browser.
CO2 AT3 Continuity	Use the same capstone title and infrastructure direction already defended in CO2 AT3. Do not repeat VM sizing; focus on cloud service access.

Learn

A cloud-backed application is accessed through a browser or mobile client. The user action travels through frontend, API/backend, authentication, platform services, storage/database and monitoring/logging components. GitHub Pages is the primary beginner path because it can host a simple static page from a repository. Firebase Hosting may be used when the student is comfortable with Firebase CLI or Cloud Shell.

Do

☐ Create or open a GitHub repository for CO3 AT1. Add a simple index.html file. The page should show the capstone title, one user action and one cloud service used. Example page sections: product name, user action, API endpoint idea, storage used and evidence link.

☐ In the repository settings, open Pages. Select the main branch and root folder as the publishing source when available. Save the Pages setting and wait for GitHub to publish the site. Open the generated Pages URL in the browser and verify that the page loads.

☐ If Firebase Hosting is used instead, open Firebase Console, choose Hosting, follow the hosting setup guidance, and deploy using Firebase CLI or Cloud Shell only when faculty confirms that the device and account are ready. For most beginners, GitHub Pages evidence is sufficient for this experiment.

☐ Use Postman evidence from Experiment 2 or create a new test request that represents one action from the hosted page. Example: the user clicks Submit Complaint, the frontend sends POST /reports, the backend stores the complaint image path and returns a status response.

☐ Draw the complete access workflow in draw.io: Browser/Mobile Client -> Hosted Frontend -> API Endpoint -> Authentication Check -> Backend/Serverless Logic -> Cloud Storage/Database -> Response -> Monitoring/Log. Paste the hosted URL, browser screenshot, workflow diagram and GitHub repository link.

Observe

Verify whether the hosted page or simulated access URL opens, whether the API/storage flow is technically consistent, and whether monitoring/logging evidence is identified.

Record Evidence

Evidence Item	Student Reference / Screenshot Number	Checkpoint
Hosted page URL or approved simulation evidence.	https://tharu-ui.github.io/Cloud-Computing-and-BDA/PulsePay/CO3-AT1-Cloud-Access-Experiments/index.html	[x] Completed
Browser access screenshot.	01-github-pages-live.png	[x] Completed
API/storage workflow diagram.	API/storage workflow diagram.png	[x] Completed
Monitoring/logging metric table.	Monitoring/logging metric table.png	[x] Completed

GitHub repository link.	https://github.com/tharu-ui/Cloud-Computing-and-BDA/tree/main/PulsePay/CO3-AT1-Cloud-Access-Experiments	[x] Completed

Explain

Explain the complete request flow from client action to cloud output and monitoring.

Student Explanation Space

The request flow starts when a merchant clicks "Verify Screenshot" in the hosted frontend. The frontend sends this as an HTTP request to POST /verify. The API Gateway checks authentication (Firebase Auth token) before passing the request to the backend's serverless logic. The Cloud Function reads/writes to Cloud Storage (image) and Firestore (record), runs OCR and fraud scoring, and returns a structured JSON response displayed to the merchant. Cloud Monitoring and Logging capture request counts, errors, and latency throughout.

Extend to Capstone

Extend the workflow to the capstone product carried forward from CO2 AT3, showing how the planned infrastructure now exposes usable cloud services.

Capstone Title	PulsePay: Fake screenshot detection using Cloud computing
CO2 AT3 Infrastructure Decision Carried Forward	Serverless-first architecture: Firebase Hosting, API Gateway + Cloud Functions, Firestore + Cloud Storage
Cloud Service / Access Layer Added in this Experiment	Hosted frontend (GitHub Pages) demonstrating the full client-to-cloud access path
Risk or Limitation Identified	This is a static demo page; the real /verify endpoint and auth flow still need to be built and connected
Improvement for Prototype or Pilot	Connect the hosted frontend to a real deployed Cloud Function endpoint and add Cloud Monitoring dashboards

Submit

GitHub Folder	CO3_AT1/Experiment_5
Files to Upload	Completed PDF, diagrams, screenshots and any exported table.
Google Classroom Entry	Paste GitHub repository link and attach final PDF.

Final Submission Checklist

Checklist Item	Status
Experiment 1 completed with storage model evidence	[x]

Experiment 2 completed with API evidence	[x]
Experiment 3 completed with serverless/event-flow evidence	[x]
Experiment 4 completed with edge/fog/cloud reasoning	[x]
Experiment 5 completed with cloud application access workflow	[x]
All diagrams exported and pasted	[x]
All screenshots are readable and privacy-safe	[x]
Capstone extension completed for every experiment	[x]
GitHub folder created and link added	[x]
Google Classroom submission completed	[x]

Student Assurance

Declaration	I confirm that this CO3 AT1 guided experiment workbook contains my own cloud access evidence, diagrams, observations, explanations and capstone extensions. I have not shared private credentials or sensitive information.
Student Signature	D.S.Tharuniya Priya
Faculty Verification	
Date	